

Efficient Fine-Tuning of Code Language Models Under Hardware Constraints: A Comparative Study of Single-GPU QLoRA and Dual-GPU DeepSpeed ZeRO-2

Talha Rashid*, Bilal Fayyaz[†]
Department of Computer Science
Roll No: 21I-1365*, 22I-1992[†]

Abstract—Fine-tuning language models for code generation is frequently limited by GPU memory, runtime, and distributed training complexity. This work presents a practical systems comparison between two efficient fine-tuning strategies for the same code-instruction task: single-GPU QLoRA on Google Colab using one Tesla T4 GPU, and dual-GPU DeepSpeed ZeRO-2 full fine-tuning on RunPod using two NVIDIA RTX A4000 GPUs. Both experiments used the Qwen2.5-0.5B-Instruct model and a 1,000-sample subset of the CodeAlpaca 20K dataset, split into 900 training and 100 evaluation samples. The QLoRA run trained only 8.80M parameters, corresponding to 1.7497% of the 502.83M-parameter model, and achieved a final training loss of 0.448 and evaluation loss of 0.823 over five epochs. The DeepSpeed ZeRO-2 run trained all 494.03M parameters, completed one epoch with a training runtime of 247.09 s, achieved a training loss of 0.792 and evaluation loss of 0.616, and reached 3.642 training samples per second across two GPUs. The results show that QLoRA is highly suitable for low-memory single-GPU fine-tuning, whereas DeepSpeed ZeRO-2 provides stronger throughput and full-model adaptation when multi-GPU hardware is available.

Index Terms—QLoRA, LoRA, DeepSpeed, ZeRO-2, large language models, fine-tuning, CodeAlpaca, GPU utilization, parameter-efficient fine-tuning.

I. INTRODUCTION

Large language models (LLMs) have become widely used for code generation, reasoning, and instruction following. However, adapting these models to a specific task or domain remains expensive because full-parameter fine-tuning can require large amounts of GPU memory and careful distributed training configuration. These constraints are especially relevant in student and research environments where access may be limited to a single cloud notebook GPU or a temporary rented multi-GPU instance.

The original objective of this project was to implement an optimized fine-tuning pipeline for a code instruction model under limited hardware. In practice, the work evolved into a two-stage comparison. First, the model was fine-tuned on Google Colab using a single Tesla T4 GPU with QLoRA-style parameter-efficient training. Second, the same dataset and model family were used on RunPod with two RTX A4000 GPUs and DeepSpeed ZeRO-2 full-parameter training.

This paper reports the completed metrics from both experiments. The focus is not to introduce a new fine-tuning algorithm, but to evaluate the practical trade-offs between adapter-based single-GPU training and distributed full-model training. The main research question is:

How do single-GPU QLoRA and dual-GPU DeepSpeed ZeRO-2 differ in trainable parameter count, runtime, memory use, throughput, loss, and hardware utilization for the same code instruction tuning task?

II. RELATED WORK

A. LoRA and QLoRA

LoRA freezes the pretrained model weights and injects low-rank trainable matrices into selected transformer layers, reducing the number of trainable parameters required for downstream adaptation [1]. QLoRA extends this idea by loading the base model in 4-bit quantized form while back-propagating into LoRA adapters, which substantially reduces memory requirements for fine-tuning [2]. This makes QLoRA particularly useful when training must be completed on a single commodity GPU.

B. DeepSpeed ZeRO Optimization

DeepSpeed ZeRO reduces memory redundancy in distributed training by partitioning optimizer states, gradients, and in later stages, model parameters across data-parallel workers [3]. ZeRO-2 partitions optimizer states and gradients, making it a practical option for multi-GPU full-parameter fine-tuning while retaining the ability to update the entire model. In this work, ZeRO-2 is used to evaluate full fine-tuning across two RTX A4000 GPUs.

C. Code Instruction Fine-Tuning

The CodeAlpaca 20K dataset provides prompt-completion examples for programming and instruction-following tasks [6]. The dataset version used in this work contains `prompt` and `completion` fields. Each example was reformatted into a structured question-reasoning-answer representation to align with the project objective of producing explicit response structure.

III. METHODOLOGY

A. Dataset Formatting

The dataset was loaded as HuggingFaceH4/CodeAlpaca_20K. A 1,000-sample subset was selected using a fixed seed of 42, producing 900 training samples and 100 evaluation samples. Each example was transformed as:

$$T_i = [Q_i; R_i; A_i] \quad (1)$$

where Q_i is the original prompt, A_i is the original completion, and R_i is a short synthetic reasoning template. The final training string used the following form:

```
### Question: prompt
### Reasoning: reasoning template
### Answer: completion
```

The maximum sequence length was 512 tokens in both experimental tracks.

B. Single-GPU QLoRA Pipeline

The single-GPU experiment used Qwen/Qwen2.5-0.5B-Instruct as the base model. The model was loaded using 4-bit quantization with LoRA adapters applied to transformer attention and feed-forward projection layers. The target modules included `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`. The LoRA rank was 16, alpha was 32, and dropout was 0.05. Training used FP16, a per-device batch size of 1, gradient accumulation of 8, and five epochs.

C. Dual-GPU DeepSpeed ZeRO-2 Pipeline

The DeepSpeed experiment used the same model and 900/100 train-evaluation split but performed full-parameter training. The RunPod instance provided two NVIDIA RTX A4000 GPUs. DeepSpeed ZeRO-2 was configured with FP16 training, gradient accumulation, reduce-scatter, all-gather partitioning, and contiguous gradients. Unlike the QLoRA run, all model parameters were trainable.

Training telemetry was collected using `nvidia-smi`, while final training and evaluation metrics were collected from the Hugging Face Trainer state and DeepSpeed output files. The completed DeepSpeed run reached a global step count of 57 for one epoch.

IV. EXPERIMENTAL SETUP

A. Hardware Environments

Table I summarizes the two execution environments.

B. Evaluation Metrics

The comparison uses trainable parameter count, trainable percentage, training runtime, throughput, training loss, evaluation loss, peak memory, total floating point operations, and GPU utilization. The trainable percentage is computed as:

$$P_{trainable} = \frac{N_{trainable}}{N_{total}} \times 100. \quad (2)$$

TABLE I: Hardware and Training Environment

Item	Single-GPU QLoRA	DeepSpeed ZeRO-2
Platform	Google Colab	RunPod
GPU	1 × Tesla T4	2 × RTX A4000
Training type	Adapter tuning	Full fine-tuning
Precision	4-bit + FP16	FP16
Distributed	No	Yes
Dataset split	900 / 100	900 / 100
Max sequence length	512	512
Epochs	5	1

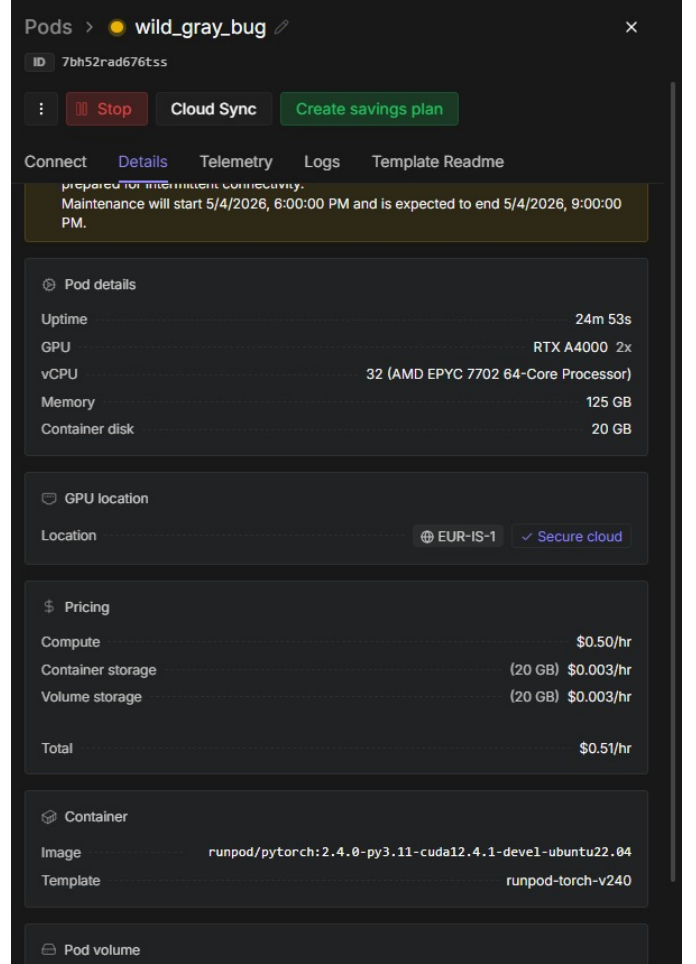


Fig. 1: RunPod environment showing the dual RTX A4000 GPU pod used for the DeepSpeed ZeRO-2 experiment.

V. RESULTS

A. Single-GPU QLoRA Results

The QLoRA run completed successfully on a Tesla T4 GPU. It trained 8,798,208 parameters out of 502,830,976 total parameters, corresponding to 1.7497% trainable parameters. The final training loss was 0.448 and the evaluation loss was 0.823. The training runtime was 2,829.545 s, with throughput of 1.59 samples/s and 0.20 steps/s. Peak GPU memory allocated was 1.66 GB and peak reserved memory was 3.56 GB.

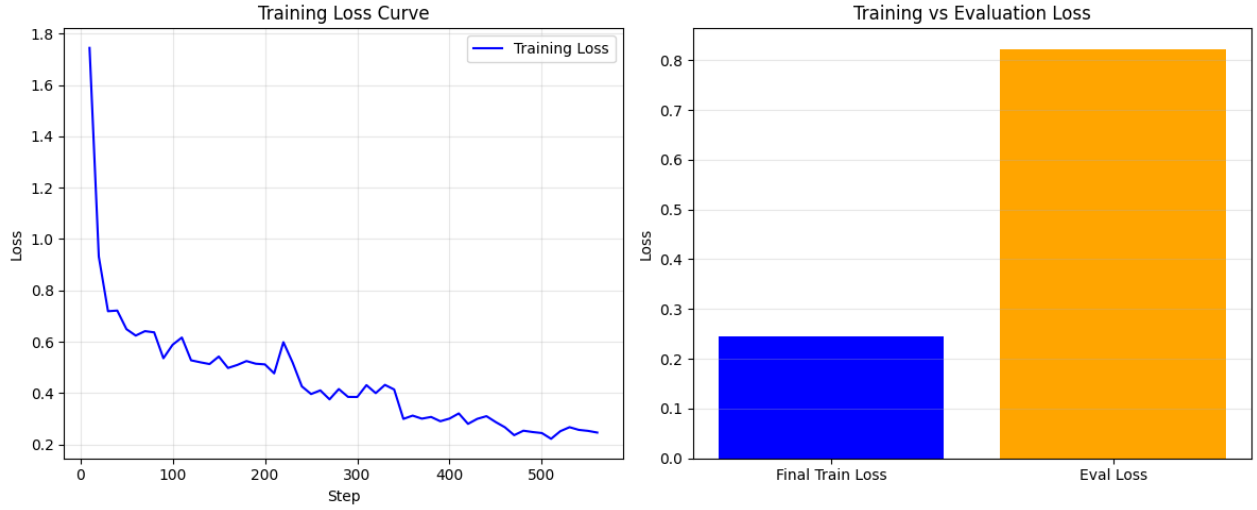


Fig. 2: Training loss curve and final train/evaluation loss visualization from the single-GPU QLoRA notebook.

TABLE II: Single-GPU QLoRA Metrics from Final Notebook Run

Metric	Value
GPU	Tesla T4
Train samples	900
Eval samples	100
Epochs	5
Global steps	565
Trainable parameters	8,798,208
Total parameters	502,830,976
Trainable percentage	1.7497%
Train runtime	2,829.545 s
Train samples/s	1.590
Train steps/s	0.200
Train loss	0.448
Eval loss	0.823
Peak allocated memory	1.66 GB
Peak reserved memory	3.56 GB
Total FLOPs	1.204×10^{15}

TABLE III: Dual-GPU DeepSpeed ZeRO-2 Completed Metrics

Metric	Value
GPU setup	$2 \times$ RTX A4000
Train samples	900
Eval samples	100
Epochs	1
Global steps	57
Trainable parameters	494,032,768
Total parameters	494,032,768
Trainable percentage	100.0000%
Train runtime	247.091 s
Manual wall time	249.900 s
Train samples/s	3.642
Train steps/s	0.231
Train loss	0.792
Eval loss	0.616
Eval runtime	2.852 s
Eval samples/s	35.058
Peak allocated memory	5.9008 GB
Peak reserved memory	9.3398 GB
Total FLOPs	9.895×10^{14}

B. Dual-GPU DeepSpeed ZeRO-2 Results

The DeepSpeed ZeRO-2 run completed successfully across two RTX A4000 GPUs. It trained all 494,032,768 parameters, corresponding to 100% trainable parameters. The run completed one epoch with 57 global optimization steps. The training runtime was 247.091 s, while manually measured wall-clock time was 249.900 s. Training throughput reached 3.642 samples/s and 0.231 steps/s. The final training loss was 0.792 and the evaluation loss was 0.616.

C. Comparison

Table IV compares the completed QLoRA and DeepSpeed experiments. QLoRA is more parameter-efficient, while DeepSpeed provides faster runtime and better evaluation loss for the one-epoch full-parameter run.

D. GPU Telemetry

The completed DeepSpeed run showed high GPU activity during training. The user-captured `nvidia-smi` snapshot

```

Every 0.1s: nvidia-smi
e4fe595897a: Sun May 3 17:22:30 2026
Sun May 3 17:22:33 2026
+-----+-----+-----+-----+-----+-----+
| NVIDIA-SMI 550.127.08 | Driver Version: 550.127.08 | CUDA Version: 12.4 |
+-----+-----+-----+-----+-----+-----+
| GPU Name | Perf | Persistence-M | Bus-Id | Disp.A | Volatile Uncorr. ECC |
| Fan Temp | Pwr:Usage/Cap | | | | |
+-----+-----+-----+-----+-----+-----+
| 0 NVIDIA RTX A4000 | 41% 44C P2 | 51W / 140W | On | 00000000:01:00:0 Off | 100% Default Off |
| 41% 44C P2 | 51W / 140W | 1329MiB / 16376MiB | 100% Default Off |
+-----+-----+-----+-----+-----+-----+
| 1 NVIDIA RTX A4000 | 41% 41C P2 | 51W / 140W | On | 00000000:C1:00:0 Off | 100% Default Off |
| 41% 41C P2 | 51W / 140W | 1329MiB / 16376MiB | 100% Default Off |
+-----+-----+-----+-----+-----+-----+

Processes:
+-----+-----+-----+-----+-----+-----+
| GPU | GI | CI | PID | Type | Process name | GPU Memory |
| ID | ID | | | | | Usage |
+-----+-----+-----+-----+-----+-----+

```

Fig. 3: RunPod `nvidia-smi` snapshot showing both RTX A4000 GPUs active at 100% utilization during the DeepSpeed ZeRO-2 experiment.

TABLE IV: Completed Comparison of Single-GPU QLoRA and Dual-GPU DeepSpeed ZeRO-2

Metric	Single-GPU QLoRA	Dual-GPU DeepSpeed ZeRO-2
Hardware	1 $\times$ Tesla T4	2 $\times$ RTX A4000
Fine-tuning type	4-bit LoRA adapter tuning	Full-parameter fine-tuning
Trainable parameters	8.80M	494.03M
Trainable percentage	1.7497%	100.0000%
Dataset split	900 train / 100 eval	900 train / 100 eval
Epochs	5	1
Global steps	565	57
Training runtime	2,829.545 s	247.091 s
Training samples/s	1.590	3.642
Training steps/s	0.200	0.231
Training loss	0.448	0.792
Evaluation loss	0.823	0.616
Peak allocated memory	1.66 GB	5.9008 GB per rank
Peak reserved memory	3.56 GB	9.3398 GB per rank
Total FLOPs	1.204×10^{15}	9.895×10^{14}

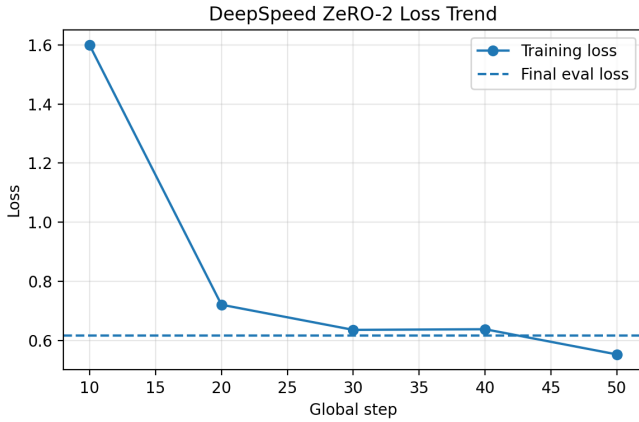


Fig. 4: DeepSpeed ZeRO-2 training loss trend across logged global steps, with the final evaluation loss shown as a dashed reference line.

shows both A4000 GPUs at 100% utilization, using approximately 1329 MiB per GPU at that instant. The Trainer-reported peak memory was higher, with 5.9008 GB allocated and 9.3398 GB reserved per rank, which is expected because instantaneous `nvidia-smi` snapshots and PyTorch peak memory counters measure different aspects of memory behavior.

VI. DISCUSSION

The experiments demonstrate two distinct forms of efficiency. QLoRA is efficient because it updates only a small adapter subset of the model. It trained 1.7497% of the parameters and remained within a low memory footprint on a Tesla T4. This makes it attractive for low-cost experimentation, classroom assignments, and rapid prototyping.

DeepSpeed ZeRO-2 is efficient in a different way. It uses more memory per rank and trains 100% of the model parameters, but it completes the 900-sample training run much faster than the single-GPU QLoRA run. The DeepSpeed training runtime was 247.091 s compared with 2,829.545 s for the

QLoRA run. This runtime difference must be interpreted carefully because the QLoRA run used five epochs, whereas the DeepSpeed run used one epoch. However, the throughput comparison is still useful: DeepSpeed reached 3.642 samples/s, compared with 1.590 samples/s for QLoRA.

The loss metrics show a mixed but meaningful outcome. QLoRA achieved lower training loss, which is partly expected because it trained for five epochs. DeepSpeed achieved lower evaluation loss after one epoch, suggesting that full-parameter updates can adapt the model more broadly when sufficient GPU resources are available. However, the two methods are not strictly identical in optimization path, trainable parameter count, epoch count, or hardware.

VII. LIMITATIONS

This study has four main limitations. First, the QLoRA and DeepSpeed runs used different hardware, so runtime is not a hardware-neutral comparison. Second, the QLoRA run used five epochs while the DeepSpeed run used one epoch. Third, the reasoning field was synthetically inserted because CodeAlpaca provides prompt-completion pairs rather than verified reasoning traces. Fourth, quality was evaluated primarily through loss and a limited qualitative code-generation test rather than a full benchmark suite such as HumanEval or MBPP.

VIII. CONCLUSION

This paper presented a practical comparison between single-GPU QLoRA and dual-GPU DeepSpeed ZeRO-2 for code instruction tuning. The QLoRA experiment successfully fine-tuned Qwen2.5-0.5B-Instruct on a Tesla T4 by training only 8.80M adapter parameters, achieving a final training loss of 0.448 and evaluation loss of 0.823. The DeepSpeed ZeRO-2 experiment completed one epoch of full-parameter training on two RTX A4000 GPUs, training all 494.03M parameters with a runtime of 247.091 s, training throughput of 3.642 samples/s, and evaluation loss of 0.616. Overall, QLoRA is the stronger choice for constrained single-GPU environments, while DeepSpeed ZeRO-2 is better suited for faster distributed full-model fine-tuning when multi-GPU resources are available.

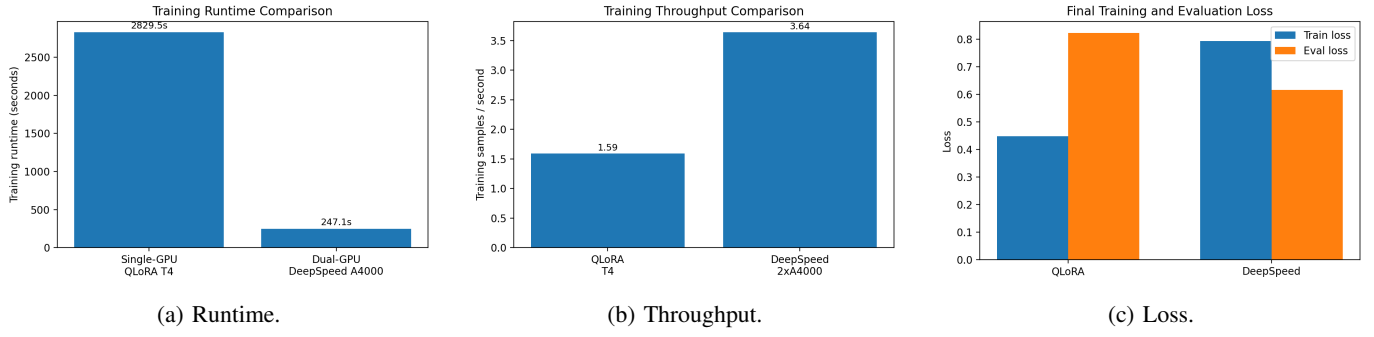


Fig. 5: Completed comparison of runtime, throughput, and loss for the single-GPU QLoRA and dual-GPU DeepSpeed ZeRO-2 runs.

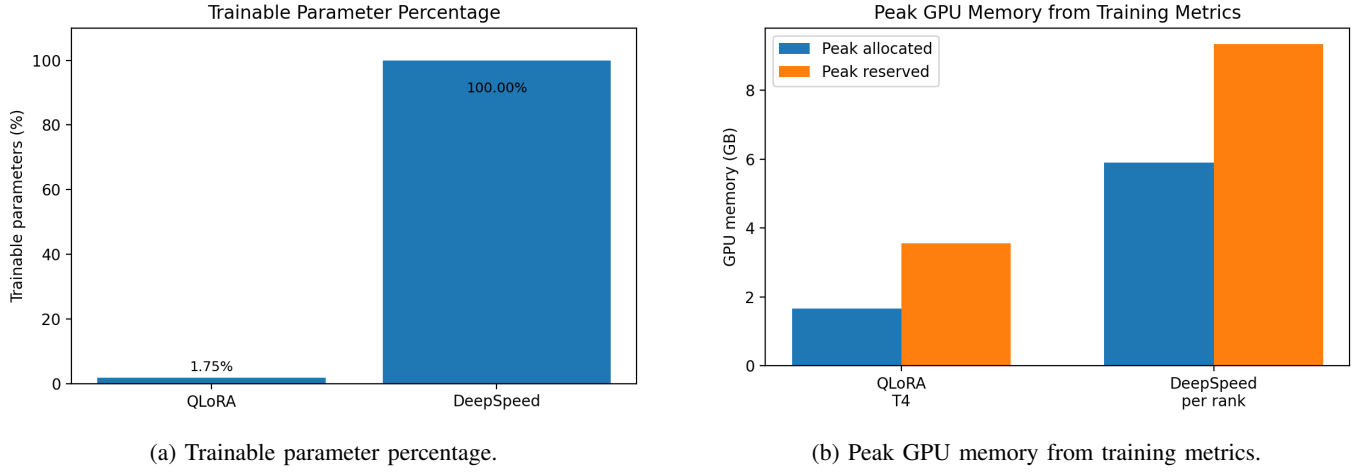


Fig. 6: Parameter-efficiency and memory trade-off between QLoRA and DeepSpeed ZeRO-2.

Future work should repeat the experiments with equal epoch counts, add larger data subsets such as 5,000 or 10,000 samples, and evaluate generated code using standardized benchmarks. A scaled DeepSpeed experiment with ZeRO-3 could further clarify the memory and communication trade-offs for larger models.

REFERENCES

- [1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proc. International Conference on Learning Representations (ICLR)*, 2022.
- [2] T. Detrmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” in *Advances in Neural Information Processing Systems*, 2023.
- [3] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.
- [4] Microsoft, “DeepSpeed: Extreme-scale model training for everyone,” DeepSpeed documentation. [Online]. Available: <https://www.deepspeed.ai/>
- [5] Qwen Team, “Qwen2.5-0.5B-Instruct,” model documentation. [Online]. Available: <https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct>
- [6] Hugging Face H4, “CodeAlpaca 20K Dataset,” dataset card. [Online]. Available: https://huggingface.co/datasets/HuggingFaceH4/CodeAlpaca_20K
- [7] T. Wolf *et al.*, “Transformers: State-of-the-Art Natural Language Processing,” in *Proc. EMNLP: System Demonstrations*, 2020.
- [8] Hugging Face, “PEFT: Parameter-Efficient Fine-Tuning,” software documentation. [Online]. Available: <https://huggingface.co/docs/peft/>